

Reviewer Pack — Minimal Frobenius Normal Form Dimension and Resolution Proof...

Entry b712cdd1c010 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 18:14:18 UTC

Minimal Frobenius Normal Form Dimension and Resolution Proof Width Inequality

Entry ID: b712cdd1c010 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 18:14:18 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Frobenius normal form) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

For any given CNF formula φ with n variables, the dimension of the Frobenius normal form matrix $F(\varphi)$ associated with φ is linearly correlated with its resolution proof width $w(\varphi)$, such that $\dim(F(\varphi)) = \Omega(w(\varphi))$.

Rationale (proposer's reasoning):

The Frobenius normal form provides a representation-theoretic structure for matrices over finite fields, which could potentially reveal deeper connections between algebraic structures and the complexity of resolution proofs. If the dimension is correlated with proof width, it suggests that the structure encodes information relevant to proof complexity.

Taxonomy category: algebraic-complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c01100d2cc69dfe4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For CNF formulas φ , if the Pearson's correlation coefficient r of $\dim(F(\varphi))$ and $w(\varphi)$ is ≥ 0.7 over a sample of at least 30 formulas, then support the conjecture; otherwise falsify it.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Frobenius normal form" AND "resolution proof complexity" - "dimension of Frobenius normal form" AND CNF formula" - "linear correlation" AND "Frobenius normal form" AND resolution

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

# Helper functions for matrix operations
def matrix_multiply(A, B):
    if len(A[0]) != len(B):
        raise ValueError("Incompatible dimensions for matrix multiplication")
    result = [[sum(a * b for a, b in zip(row_a, col_b)) for col_b in zip(*B)] for row_a in A]
    return result

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        # Find the pivot
        max_row = i + max(range(i, rows), key=lambda k: abs(matrix[k][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below the pivot
        for j in range(i + 1, rows):
            factor = Fraction(matrix[j][i], matrix[i][i])
            for k in range(cols):
                matrix[j][k] -= factor * matrix[i][k]

    return matrix

def rank(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rref = gaussian_elimination(matrix)
    rank = 0
    for row in rref:
        if any(row):
            rank += 1
```

```

    return rank

# Function to generate a random CNF formula with n variables and m clauses
def generate_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = [random.randint(1, n) * (-1 if random.choice([True, False]) else 1) for _ in range(n)]
        cnf.append(clause)
    return cnf

# Function to compute the resolution proof width of a CNF formula
def resolution_width(cnf):
    clauses = set(tuple(sorted(c)) for c in cnf)
    queue = list(clauses)
    seen = set(queue)
    while queue:
        clause = queue.pop(0)
        if len(clause) == 1:
            return abs(clause[0])
        for other in queue:
            for i in range(len(other)):
                if -other[i] in clause:
                    new_clause = tuple(sorted(set(other[:i] + other[i+1:])))
                    if new_clause not in seen:
                        seen.add(new_clause)
                        queue.append(new_clause)
    return float('inf')

# Function to run one trial
def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([5, 10, 15, 20, 30, 40])
    m = random.randint(2 * n, 5 * n)
    cnf = generate_cnf(n, m)

    dim_F_phi = rank([[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)])
    w_phi = resolution_width(cnf)

    return {
        "metric_name": "Dimension of Frobenius Normal Form",
        "metric_value": dim_F_phi,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": dim_F_phi >= 0.7 * w_phi,
        "counterexample": f"dim(F(φ))={dim_F_phi}, w(φ)={w_phi}" if not dim_F_phi >= 0.7 * w_phi else ""
    }

# Main function to run multiple trials
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

    results.append(result)
    print(f"TRIAL: {result}")

metric_values = [r["metric_value"] for r in results if r["instances_tested"] > 0]
conjecture_holds = all(r["conjecture_holds"] for r in results if r["instances_tested"] > 0)

if conjecture_holds:
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = len([r for r in results if r["conjecture_holds"]]) / len(results)
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1255b3f.py", line 102, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1255b3f.py", line 83, in run_trial
    dim_F_phi = rank([[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)])
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1255b3f.py", line 42, in rank
    rref = gaussian_elimination(matrix)
         ^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f1255b3f.py", line 34, in gaussian_elimination
    factor = Fraction(matrix[j][i], matrix[i][i])
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(0, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the Pearson's correlation coefficient could not be calculated and the sample size was not reached. | next: Re-run the test with a larger sample size to ensure the support condition is met. Investigate the cause of the crash to prevent future occurrences.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12691
2	propose	ollama_remote	glm4:latest	0	0	12494
3	preregistration	ollama_remote	glm4:latest	0	0	8942
4	novelty	ollama_remote	glm4:latest	0	0	9025
5	novelty	ollama_remote	glm4:latest	0	0	9516
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40090
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10511
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9712
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12667
10	judge	ollama_remote	glm4:latest	0	0	20794

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 146442 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/b712cdd1c010.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b712cdd1c010.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b712cdd1c010.tar.gz` (if generated)